Procesadores del Lenguaje
Curso 2025-2026
Práctica Final
Traductor frontend: subconjunto de lenguaje C a Lisp (código intermedio)
Traducción backend: Lisp a notación postfija (código final)

Pablo García Aparicio - 100522190
Miguel Merino Sánchez - 100522156



ÍNDICE

1. Declaración de Uso de IA y Participación de Autores....3
2. Introducción....3
Desarrollo del Frontend (Traductor de C a Lisp)....4
1. Gestión de Ámbitos: Variables Locales y Globales....4
2. La Función MAIN....4
3. Impresión de Cadenas ("puts")....5
4. Impresión de Expresiones ("printf")....5
5. Operadores, Precedencia y Asociatividad....6
6. Estructura de Control ("while")....6
7. Estructura de Control ("if")....6
8. Variables Locales y Gestión de Ámbitos....7
9. Estructura de Control ("for")....7
10. Estructura de Selección ("switch")....8
11. Definición y Llamada de Funciones....8
12. Implementación de Vectores (Arrays)....9
Desarrollo del Backend (Traductor de C a Lisp)....10
13. Backend: Traducción Directa a Notación Postfija....10
14. Backend: Gestión de Variables y Memoria....10
15. Backend: Estructuras de Control ("while" e "if")....11
16. Backend: Función Principal e Impresión por Pantalla....11
ANEXO: Pruebas propias....12
1. Pruebas enlazadas (Frontend y Backend)....12
Prueba A: Main, impresión y funcionamiento de variables globales....12
Prueba B: Operadores, precedencia y asociatividad....13
Prueba C: Uso de While + IF....14
2. Pruebas de Compilación de Alto Nivel (Frontend)....15
Prueba D: Gestión de Ámbitos Internos y bucles For....15
Prueba E: Selección Condicional Múltiple....16
Prueba F: Subrutinas de Usuario, Retornos y Parámetros....17
Prueba F: Arrays y vectores....18

1. Declaración de Uso de IA y Participación de Autores

Los abajo firmantes, Pablo García Aparicio y Miguel Merino Sánchez, estudiantes de la Universidad Carlos III de Madrid, declaramos formalmente que el trabajo presentado en esta práctica ha sido realizado mediante la contribución equitativa de ambos miembros del equipo.

Asimismo, en cumplimiento con la normativa de Integridad Académica de la UC3M, declaramos bajo nuestra responsabilidad que:

- El código fuente que hemos entregado es propio de nuestro trabajo personal. No se han utilizado herramientas de Inteligencia Artificial Generativa para la creación, redacción o resolución automática del código ni de la lógica del programa, garantizando que el desarrollo intelectual del proyecto nos corresponde a nosotros
- El trabajo se ajusta a las directrices de uso ético de herramientas tecnológicas establecidas por la universidad en la guía de IA Generativa y en el Reglamento General de la UC3M

2. Introducción

El propósito fundamental de este proyecto es el diseño, desarrollo e implementación completa de un Procesador de Lenguaje en dos etapas diferenciadas, estructurado como un compilador pipeline que traslada un lenguaje de alto nivel a un lenguaje de bajo nivel orientado a pila. El sistema se compone de dos herramientas independientes pero conectadas en cascada:

1. Frontend (trad3.y): Actúa como un analizador léxico y sintáctico avanzado que toma como entrada un subconjunto específico del lenguaje de programación C (con tipos enteros, estructuras de control completas, funciones, arrays unidimensionales, etc.) y lo traduce a un código intermedio equivalente en el paradigma funcional de Common Lisp.

2. Backend (back3.y): Actúa como la segunda fase de compilación. Toma el código Lisp generado por el frontend (expresiones matemáticas, lógicas, sentencias globales e iterativas) y lo transforma a notación postfija pura para ser ejecutado en el intérprete

Desarrollo del Frontend (Traductor de C a Lisp)

El objetivo principal ha sido el diseño de un traductor dirigido por sintaxis usando Bison, este es capaz de convertir un subconjunto de un lenguaje(C) en codigo intermedio Lisp. Para no cometer un consumo exponencial de memoria y bloqueos, hemos elegido el generar e imprimir el codigo Lisp a nivel de función y sentencia, con la intención de evitar acumular toda la traducción en el axioma principal

Ahora detallaremos las decisiones de diseño y las reglas gramaticales implementadas para poder cumplir con las especificaciones del enunciado:

1. Gestión de Ámbitos: Variables Locales y Globales

Para empezar con el traductor, nos centramos en cómo podemos guardar datos al inicio del programa. En C es muy común declarar una variable y dejarla vacía, pero si hacemos eso directamente en Lisp, el programa puede dar errores al intentar usarla después. Para evitar problemas futuros decidimos que nuestro traductor les asigne un cero por defecto de manera automática.

Como en C se pueden declarar muchísimas cosas en una sola línea (separadas por comas), creamos una regla recursiva (lista_ids) que va leyendo los elementos uno a uno. Según lo que se encuentre el analizador, aplicamos una solución distinta:

- **Variables básicas (int a;):** Si no hay valor inicial, el traductor no se arriesga y le asigna un cero, generando (setq a 0).
- **Variables con valor (int a = 5;):** Si el programador ya le ha dado un valor constante, lo capturamos y lo pasamos tal cual, generando (setq a 5).

Un detalle clave de este diseño es que vamos imprimiendo estas traducciones sobre la marcha. Así, evitamos que Bison tenga que almacenar cadenas de texto muy largas en su memoria interna, ahorrando recursos y manteniendo intacto el orden original del programa.

2. La Función MAIN

Esta función es el punto de entrada necesario en todo programa escrito en C. Nuestra idea es que el analizador lea primero las declaraciones sueltas y termine siempre, de manera obligatoria, procesando la función principal.

Para conseguir esto, modificamos la regla principal de nuestra gramática, a la que llamamos el "axioma". La preparamos para que procese primero cualquier lista de variables o funciones personalizadas (apoyándose en la regla "declaraciones_y_funciones") y, justo después, exija la aparición de la regla "funcion_main".

El proceso en el que se analiza esta parte lo dividimos en un par de acciones clave:

- Captura y transformación: Al detectar la sintaxis exacta de "main () { ... }", el traductor coge todo el bloque de código que se ha analizado en su interior (gracias a nuestra regla "lista_sentencias") y lo envuelve usando la sintaxis propia de LISP para definir funciones. El resultado final que imprimimos es el bloque "(defun main () ...)".
- Gestión interna: Aprovechamos el momento exacto en el que se abre la llave "{" para realizar dos tareas ocultas en el código de C. Por un lado, le indicamos al traductor que el nombre de la función actual es "main". Por otro, reiniciamos a cero el contador de nuestra tabla de variables locales ("n_local = 0").

Este último detalle de poner el contador a cero es una pequeña implementación que nos facilitará muchísimo la vida en los siguientes pasos, ya que nos garantiza que las variables declaradas dentro de distintas funciones no se van a mezclar ni a pisar entre sí.

3. Impresión de Cadenas ("puts")

Para comprobar que el traductor funcionaba, necesitábamos que mostrara texto por pantalla. En el lenguaje C, la instrucción más directa para imprimir un mensaje con un salto de línea es "puts". Como la función "print" de Lisp se comporta exactamente igual, la traducción fue muy directa.

Hemos dividido la solución en dos pequeños pasos:

1. **Reconocimiento del texto**: Modificamos el analizador léxico para que, al detectar unas comillas dobles, lea todo el texto de su interior y lo agrupe bajo el identificador "STRING". Además, incluimos "puts" en nuestra lista de palabras reservadas.
2. **Traducción al nuevo lenguaje:** En la gramática principal, creamos una variante que reconoce el patrón exacto: la palabra "puts" seguida del texto entre paréntesis. Como el texto capturado ya conserva sus propias comillas originales, simplemente lo incrustamos en la orden de Lisp.

De esta forma, el programa genera automáticamente la instrucción "(print "texto_del_usuario")", logrando que la terminal muestre el mensaje original sin complicaciones añadidas.

4. Impresión de Expresiones ("printf")

Dar el salto a "printf" era necesario para poder imprimir variables mezcladas con texto, pero intentar que nuestro analizador procesara la típica cadena de formato (como "%d") complicaba el diseño en exceso. Para solucionarlo de forma práctica, aplicamos la siguiente lógica:

- Ignorar el formato inicial: La gramática reconoce el primer texto del "printf" para que la sintaxis de C sea válida y no dé error, pero a la hora de generar la traducción a Lisp, lo descartamos por completo.
- Lectura en cadena: Para el resto de argumentos (las variables o textos extra separados por comas), creamos la regla recursiva "lista_exprs", que se encarga de ir procesándolos uno por uno.

- Impresión continua con "princ": Traducimos cada uno de esos argumentos usando la función "princ" de Lisp. Elegimos esta función en lugar del clásico "print" porque imprime el valor en crudo, sin comillas extra ni saltos de línea automáticos.

5. Operadores, Precedencia y Asociatividad

Para resolver las operaciones matemáticas y lógicas sin hacer que las reglas de la gramática sean incomprensibles, nos apoyamos directamente en las herramientas nativas de Bison:

- Jerarquía de operadores: Utilizamos las directivas "%left" y "%right" en la cabecera del código para definir el orden de prioridad clásico de C (desde las sumas y restas hasta las multiplicaciones y el operador módulo).
- Operador unario: Para resolver el caso del signo negativo delante de un operando, creamos una variante en la regla "termino" y le forzamos la máxima prioridad posible utilizando la etiqueta "%prec UNARY_SIGN".
- Traducción prefija: En las acciones semánticas de la regla "expresion", convertimos la notación tradicional de C a la prefija de Lisp mediante la función "sprintf".
- Mapeo directo: Transformamos directamente los símbolos lógicos a sus equivalentes nativos de Lisp. Por ejemplo, "&&" se traduce como "and", la desigualdad "!=" se convierte en "/=" y el operador "%" pasa a ser "mod".

6. Estructura de Control ("while")

Para dotar al traductor de la capacidad de ejecutar bucles iterativos, implementamos la estructura "while" de C. El objetivo es transformar la condición y el cuerpo del bucle en una estructura compatible con Lisp sin complicar el análisis sintáctico.

El proceso se basa en los siguientes puntos:

- **Reconocimiento del patrón**: La regla "lista_sentencias" identifica el token "WHILE", captura la "expresion" condicional y procesa el bloque entre llaves.
- **Traducción semántica**: Se utiliza "sprintf" para reorganizar los elementos en la estructura "(loop while condicion do sentencias)".
- **Anidamiento**: El uso recursivo de "lista_sentencias" en el cuerpo permite incluir cualquier instrucción o bucle dentro de otro de forma natural.
- **Ajuste de separadores**: Se configuró la gramática para que los bloques terminados en llave no requieran un punto y coma (";") al final, evitando conflictos.

7. Estructura de Control ("if")

Para implementar la toma de decisiones en el flujo del programa, adaptamos la gramática para reconocer tanto el condicional simple como la variante con bloque "else".

La estructura principal se definió dentro de la regla "lista_sentencias", obligando al uso de llaves para delimitar los bloques de código y así evitar conflictos sintácticos como el del "else" colgante. Al igual que en las estructuras iterativas, configuramos el analizador para que el cierre de la llave no requiera un punto y coma final, permitiendo que la sentencia se reconozca como un bloque independiente.

La traducción semántica utiliza la función "sprintf" para reorganizar el código en la sintaxis "(if condicion bloque_then bloque_else)" de Lisp. Un aspecto fundamental en este punto fue la incorporación de la función "progn" en las acciones semánticas; esto es necesario para agrupar múltiples sentencias dentro de una misma rama del condicional, ya que el "if" de Lisp espera recibir un único elemento evaluable por cada camino. De este modo, el traductor genera estructuras del tipo "(if condicion (progn ...) (progn ...))", garantizando que toda la lógica interior se ejecute secuencialmente.

8. Variables Locales y Gestión de Ámbitos

Para que las funciones puedan tener sus propios datos sin interferir con las variables globales, implementamos un sistema de gestión de ámbitos basado en una tabla de símbolos local y el uso de prefijos. El objetivo es que Lisp no trate todas las variables como globales, respetando el encapsulamiento de cada función.

La implementación se apoya en los siguientes puntos:

- **Identificación de contexto**: Utilizamos una tabla de símbolos ("tabla_local") y funciones en C ("insertar_local" y "es_local") para registrar todas las variables declaradas dentro de una función. Al encontrar un identificador, el traductor comprueba si está en dicha tabla; si es así, le concatena el nombre de la función actual y un guion bajo (ej. "main_a") para evitar colisiones.
- **Asignación diferenciada**: Modificamos la gramática para que, una vez dentro de las funciones, las asignaciones utilicen la instrucción "setf" en lugar de "setq". Esto permite diferenciar entre la creación inicial de la variable y la modificación de su valor, asegurando que el traductor trabaje siempre sobre la variable local correcta

9. Estructura de Control ("for")

Para integrar el bucle "for" de C, hemos transformado su lógica de tres componentes en una estructura compatible con Lisp. Al no existir un constructor idéntico en el lenguaje de destino, reutilizamos la lógica del "loop while".

La implementación se basa en los siguientes puntos:

- **Distribución sintáctica**: La inicialización se extrae para ejecutarse antes del bucle, mientras que la condición se emplea como motor de parada.

- **Actualización del índice**: Las operaciones "INC" o "DEC" se insertan automáticamente al final del bloque de sentencias mediante "setf".
- **Gestión de ámbito**: El traductor verifica si la variable de control es local para añadir el prefijo de la función ("get_var_name").

10. Estructura de Selección ("switch")

Para facilitar la escritura de condicionales basados en una única variable, hemos incorporado la estructura "switch". Esta permite un flujo de control más limpio que una cadena larga de "if-else", mapeándose de forma muy natural a la instrucción "case" de Lisp.

El desarrollo se resume en los siguientes puntos:

- **Traducción de casos**: La regla "lista_casos" procesa cada bloque "case" de C, extrayendo el valor numérico y las sentencias asociadas para generar la estructura "(valor sentencias)" en Lisp.
- **Rama por defecto**: El bloque "default" de C se traduce directamente utilizando la palabra clave "otherwise" de Lisp, asegurando que siempre haya una salida si no coincide ningún valor.
- **Manejo del "break"**: Aunque en C el "break" es vital para evitar la ejecución en cascada, en nuestra traducción se descarta semánticamente. Esto se debe a que la estructura "case" de Lisp es intrínsecamente excluyente.
- **Ámbito de la variable**: Al igual que en otros puntos, el traductor verifica mediante "es_local" si la variable evaluada debe llevar el prefijo de la función actual para que el "case" funcione sobre el dato correcto.

11. Definición y Llamada de Funciones

Este apartado permite que el programador defina sus propias subrutinas con parámetros y retornos. Es el punto donde se integra toda la lógica de gestión de ámbitos para permitir que el analizador trabaje en diferentes contextos de forma segura.

La implementación se basa en los siguientes aspectos clave:

- **Cambio de contexto**: Al detectar una nueva función, el traductor utiliza la variable "nombre_funcion_actual" para actualizar el prefijo de las variables y reinicia la tabla local con "n_local = 0" para limpiar el ámbito anterior.
- **Paso de parámetros**: La regla "lista_params" registra cada argumento en la tabla local. De este modo, los parámetros reciben el mismo tratamiento que las variables locales, evitando conflictos con nombres globales.
- **Retornos estructurados**: Implementamos la sentencia "return" mediante la función "(return-from nombre_funcion expresion)". Esto permite que la función termine y devuelva el valor correcto en cualquier punto de su ejecución.

- **Llamadas a funciones**: Las llamadas se traducen a la notación prefija "(nombre_funcion argumentos)". El traductor es capaz de procesar tanto llamadas sin parámetros como aquellas que incluyen una "lista_args" de expresiones complejas.

12. Implementación de Vectores (Arrays)

Para que el traductor pudiera manejar colecciones de datos, adaptamos la gramática para soportar tanto la reserva de memoria como el acceso y la modificación de elementos. En C, los vectores se basan en punteros y corchetes, mientras que en Lisp se gestionan mediante funciones específicas que deben ser mapeadas con precisión.

El desarrollo se resume en los siguientes puntos clave:

- **Reserva de memoria**: En las declaraciones, cuando el analizador detecta el tamaño entre corchetes (ej. int a[10]), genera la instrucción (make-array tamaño). Esto reserva el espacio necesario en el entorno de Lisp de forma equivalente a C.
- **Acceso y lectura**: Para recuperar un valor (ej. x = a[i]), la regla "termino" identifica el uso de corchetes y traduce la instrucción a (aref nombre_vector indice). El índice puede ser cualquier expresión matemática compleja que el traductor evaluará previamente.
- **Asignación de elementos**: Para modificar una posición específica, el traductor genera una estructura combinada: (setf (aref nombre_vector indice) valor). Esto permite que la operación de escritura sea destructiva y actualice el array original.
- **Gestión de nombres**: Al igual que con las variables simples, el sistema utiliza "get_var_name" para comprobar si el vector es local o global, aplicando el prefijo correspondiente (como "main_vector") para evitar conflictos entre funciones.

Desarrollo del Backend (Traductor de C a Lisp)

Tras completar el *frontend*, este módulo se encarga de traducir el código intermedio Lisp a Forth. El objetivo principal ha sido transformar una sintaxis funcional y prefija en una estructura estrictamente postfija basada en pila. Para lograrlo, nos hemos apoyado nuevamente en Bison, diseñando una gramática que realiza una traducción directa y sin código diferido. A continuación, los puntos clave:

13. Backend: Traducción Directa a Notación Postfija

Para la conversión del código intermedio en Lisp (notación prefija) al código final en Forth (notación postfija), optamos por una traducción directa sin usar código diferido ni árboles de sintaxis abstracta (AST), tal y como sugieren las especificaciones.

El proceso se apoya en la siguiente lógica:

- **Análisis ascendente:** Aprovechamos el funcionamiento "bottom-up" de Bison. Al procesar las reglas de abajo hacia arriba, el analizador lee e imprime los operandos de forma natural antes de resolver la expresión completa.
- **Generación postfija automática:** En reglas como "expression", situamos el "printf" del operador matemático o lógico justo al final del patrón. De este modo, al leer la instrucción prefija de Lisp, el sistema imprime primero los valores y remata con el operador (generando la estructura "a b +").
- **Mapeo de operadores:** Implementamos la traducción directa de los operadores lógicos y relacionales a sus equivalentes exactos en Forth. Por ejemplo, el operador "not" se imprime como "0=", y la desigualdad "/=" se convierte en "= 0=".

14. Backend: Gestión de Variables y Memoria

Dado que en esta fase no se requiere el uso de variables locales ni vectores, la gestión de datos se limita a variables globales, operando directamente sobre la pila de Forth mediante sus instrucciones específicas de memoria.

El desarrollo se basa en los siguientes puntos:

- **Declaración inicial:** La regla que reconoce setq se encarga de crear la variable y darle su primer valor. Por ejemplo, (setq a 5) genera la salida variable a 5 a !.
- **Asignación (escritura):** Para actualizar un dato ya existente, la regla de setf omite la palabra de declaración y utiliza únicamente el operador de almacenamiento. Genera directamente la estructura “valor variable” !.
- **Uso como operando (lectura):** Cuando el analizador detecta que una variable forma parte de una expresión, inyecta automáticamente el operador de recuperación @ (ej. a @). Esto asegura que su valor se cargue en la cima de la pila antes de evaluar la operación.

15. Backend: Estructuras de Control ("while" e "if")

Mapeamos las sentencias de control de Lisp a las palabras reservadas de salto de Forth. El reto aquí es colocar cada etiqueta en el momento exacto para que envuelvan correctamente el código postfijo.

La implementación consta de los siguientes aspectos:

- **Bucle "while":** Aprovechando la traducción secuencial, la regla del bucle imprime BEGIN antes de evaluar la condición, WHILE antes de entrar al cuerpo del bucle, y REPEAT al finalizar, generando la estructura funcional de Forth.
- **Condicional "if":** Siguiendo la misma estrategia, la instrucción se traduce intercalando IF, ELSE y THEN entre los bloques de código previamente evaluados en postfija.
- **Prevención de conflictos:** Para evitar problemas de ambigüedad ("shift/reduce") en Bison al analizar el condicional, factorizamos la palabra reservada y la condición en una regla auxiliar llamada ifHead.

16. Backend: Función Principal e Impresión por Pantalla

Para finalizar la traducción, adaptamos la definición del punto de entrada del programa y las funciones de salida por consola, respetando la sintaxis estricta del diccionario de Forth.

El proceso se resuelve mediante los siguientes puntos:

- **Definición y llamada al "main":** Al detectar la estructura (defun main ...), el código de la función se envuelve entre los constructores : y ; para definir la palabra main en Forth. Posteriormente, su ejecución se traduce mediante una simple llamada a dicha palabra.
- **Impresión de cadenas de texto:** Las instrucciones que envían literales de texto por pantalla se traducen utilizando la palabra reservada ." " seguida de la cadena correspondiente.
- **Impresión de valores numéricos:** Cuando se necesita imprimir el resultado de una expresión o variable, el traductor inyecta el operador de impresión . (punto). Este se encarga de extraer y mostrar directamente el valor que se encuentra en la cima de la pila.
- **Control de saltos de línea:** Para mantener la fidelidad con el comportamiento original de Lisp, la regla que traduce print añade automáticamente la palabra cr al final de la instrucción para forzar el salto de línea, mientras que la regla de princ la omite.

ANEXO: Pruebas propias

Hemos creado una serie de pruebas, abarcando todos los aspectos tanto del frontend como el backend (y ambos a la vez, cuando ha sido posible) para comb¡probar y validar el

funcionamiento de nuestro programa.

1. Pruebas enlazadas (Frontend y Backend)

Prueba A: Main, impresión y funcionamiento de variables globales

Objetivo

El objetivo de esta prueba inicial es asegurarnos de que las variables globales se registran bien, ya sea asignándoles un valor o dejándolas por defecto. También comprobamos la generación de la función `main` y la traducción de primitivas como `puts` o `printf`.en, ya sea asignándoles un valor o dejándolas por defecto. También comprobamos la generación de la función `main` y la traducción de primitivas como `puts` o `printf`.

Código y traducciones:

Código de la prueba en C	Traduccion a LISP	Traduccion a Forth

`int x = 100, y, z = 50;`
 	
`main () {`
`  puts("Inicio de la prueba 1");`
`  y = x - z;`
`  printf("%d", y);`
`  puts("Fin");`
`}`
`//@ (main)`
	1 (setq x 100)
	2 (setq y 0)
	3 (setq z 50)
	4 (defun main ()
	5 (print "Inicio de la prueba 1")
	6 (setf y (- x z))
	7 (princ y)
	8 (print "Fin")
	9)
	10 (main)
	11
		pablo@pcdepablo:~/procesadores_lenguaje/practica_final$
		." Inicio de la prueba 1" cr compiled
		x @ z @ - y ! compiled
		y @ . ." Fin" cr compiled
		; ok
		main Inicio de la prueba 1
		50 Fin
		ok

Análisis y resultados:

Tras ejecutar el test, vemos que el traductor cumple su cometido a la perfección: Lisp emplea `setq` para instanciar las variables y Forth usa la directiva `variable / !`. Asimismo, la impresión del `main` en el axioma queda bloqueada como pide el enunciado y los saltos de línea de `puts` se correctamente.

Prueba B: Operadores, precedencia y asociatividad.

Objetivo

Con este test buscamos analizar la jerarquía de operaciones para corroborar que no se rompe la precedencia nativa al compilar, validando la conversión de símbolos como el módulo (`%`), la desigualdad (`!=`) o las compuertas lógicas (`&&` y `||`).

Código y traducciones:

Código de la prueba en C	Traduccion a LISP

`int val1 = 20, val2 = 3, resultado;`
 	
`main () {`
`  resultado = (val1 != val2) && (val1 % val2 == 2);`
`  printf("%d", resultado);`
`  resultado = !resultado || (val1 <= 15);`
`  printf("%d", resultado);`
`}`
`//@ (main)`
	`(setq val1 20)`
	`(setq val2 3)`
	`(setq resultado 0)`
	`(defun main ()`
	`(setf resultado (and (/= val1 val2) (= (mod val1 val2) 2)))`
	`(princ resultado)`
	`(setf resultado (or (not resultado) (<= val1 15)))`
	`(princ resultado)`
	`)`
	`(main)`

Forth:

```
pablo@pcdepablo:~/procesadores_lenguaje/practica_final$ ./trad < pruebas_propias/p
 variable val2 3 val2 !   ok
 variable resultado 0 resultado !   ok
 : main   compiled
 val1 @  val2 @  = 0=  val1 @  val2 @  mod  2  =  and  resultado !   compiled
 resultado @  .  resultado @  0=  val1 @  15  <=  or  resultado !   compiled
 resultado @  .  ;   ok
 main -1 0  ok
```

Análisis y resultados:

Al revisar la salida, se aprecia que el sistema reorganiza sin fallos la estructura aritmética y relacional. Lisp usa su notación prefija habitual, mientras que Forth apila los operandos en postfijo. Es destacable cómo la desigualdad se resuelve inteligentemente usando la igualdad invertida (`0=`) en Forth y el módulo pasa a ser `mod`.

Prueba C: Uso de While + IF

Objetivo:

Se ponen a prueba los mecanismos de iteración mediante `while` y los saltos condicionales `if-else`, comprobando que LISP inserta la función `progn` al anidar múltiples instrucciones.

Código y traducciones:

Código de la prueba en C	Traduccion a LISP

`int n = 5, suma = 0;`
 	
`main () {`
`  while (n > 0) {`
`  if (n % 2 == 0) {`
`  puts("Es par");`
`  suma = suma + n;`
`  } else {`
`  puts("Es impar");`
`  }`
`  n = n - 1;`
`  }`
`}`
`//@ (main)`
	`(setq n 5)`
	`(setq suma 0)`
	`(defun main ()`
	`(loop while (> n 0) do`
	`(if (= (mod n 2) 0)`
	`(progn`
	`(print "Es par")`
	`(setf suma (+ suma n))`
	`)`
	`(progn`
	`(print "Es impar")`
	`))`
	`(setf n (- n 1))`
	`)`
	`)`
	`(main)`

Forth:

```
pablo@pcdepablo:~/procesadores_lenguaje/practica_final$
 ;   ok
 main Es impar
Es par
Es impar
Es par
Es impar
 ok
```

Análisis y resultados:

Tras la compilación, tanto el bucle como la decisión condicional han sido traducidos sin perder el flujo lógico. La inserción de `progn` en Lisp garantiza que las ramas del `if` agrupen todas sus sentencias de forma segura, y Forth delimita correctamente las repeticiones valiéndose de los bloques `BEGIN...WHILE...REPEAT`.

2. Pruebas de Compilación de Alto Nivel (Frontend)

Prueba D: Gestión de Ámbitos Internos y bucles For

Objetivo:

En esta prueba comprobamos que las variables locales funcionan bien y no se mezclan con las de otras funciones gracias al prefijo que les añadimos. También probamos el funcionamiento del bucle `for` calculando la tabla de multiplicar del 7.

Código y traducciones:

Código de la prueba en C	Traduccion a LISP

`int global_tabla = 7;`
 	
`main () {`
`  int k;`
`  for (k = 1; k <= 5; INC(k)) {`
`  printf("%d %s %d", k, "x7=", k * global_tabla);`
`  puts("");`
`  }`
`}`
`//@ (main)`
	`(setq global_tabla 7)`
	`(defun main ()`
	`(setq main_k 0)`
	`(setf main_k 1)`
	`(loop while (<= main_k 5) do`
	`(princ main_k)(princ "x7=")(princ (* main_k global_tabla))`
	`(print "")`
	`(setf main_k (+ main_k 1))`
	`)`
	`)`
	`(main)`

Análisis y resultados:

Como se ve en la salida, el traductor convierte el `for` en un `loop while` de Lisp y coloca el incremento (`INC`) justo al final del bloque para que se repita correctamente.

Prueba E: Selección Condicional Múltiple

Objetivo:

Con este test comprobamos que nuestro traductor entiende bien la estructura `switch`. Hemos simulado una selección con diferentes códigos de estado (como 404 o 200).

Código y traducciones:

Código de la prueba en C	Traduccion a LISP

`main () {`
`  int estado = 404;`
 	
`  switch (estado) {`
`  case 200:`
`  puts("OK");`
`  break;`
`  case 404:`
`  puts("No encontrado");`
`  break;`
`  case 500:`
`  puts("Error de servidor");`
`  break;`
`  default:`
`  puts("Desconocido");`
`  break;`
`  }`
`}`
`//@ (main)`
	`(defun main ()`
	`(setq main_estado 404)`
	`(case main_estado`
	`(200 (print "OK")`
	`)(404 (print "No encontrado")`
	`)(500 (print "Error de servidor")`
	`)(otherwise (print "Desconocido")`
	`))`
	`)`
	`(main)`

Análisis y resultados:

En Lisp no existe el `switch` tal cual, así que lo hemos traducido a su equivalente `case`, ignorando los `break` de C porque en Lisp no hacen falta. El caso `default` se traduce perfectamente como `otherwise`.

Prueba F: Subrutinas de Usuario, Retornos y Parámetros

Objetivo:

Aquí queremos ver si podemos crear funciones propias pasándoles parámetros. Hemos hecho una función sencilla que calcula el área de un rectángulo.

Código y traducciones:

Código de la prueba en C	Traduccion a LISP

`int area_rectangulo (int base, int altura) {`
`  int superficie;`
`  superficie = base * altura;`
`  return superficie;`
`}`
 	
`main () {`
`  int b = 8;`
`  int h = 3;`
`  printf("%s %d", "Area:", area_rectangulo(b, h));`
`}`
`//@ (main)`
	`(defun area_rectangulo (base altura)`
	`(setq area_rectangulo_superficie 0)`
	`(setf area_rectangulo_superficie (* base altura))`
	`(return-from area_rectangulo area_rectangulo_superficie)`
	`)`
	`(defun main ()`
	`(setq main_b 8)`
	`(setq main_h 3)`
	`(princ "Area:")(princ (area_rectangulo main_b main_h))`
	`)`
	`(main)`

Análisis y resultados:

Al traducir, se comprueba que el traductor trata los parámetros como variables locales (añadiéndoles el nombre de la función delante) y que el `return` de C se transforma correctamente en un `return-from` en Lisp para devolver el resultado final.

Prueba G: Arrays y vectores

Objetivo:

El objetivo de esta última prueba es comprobar que los vectores (arrays) funcionan sin problemas. Hemos creado un pequeño array para guardar las longitudes de los lados de unos cuadrados y calcular su área.

Código y traducciones:

Código de la prueba en C:

```c
main () {
    int lados[3];
    int area_total;

    lados[0] = 3;
    lados[1] = 5;
    lados[2] = 10;

    area_total = (lados[0] * lados[0]) + (lados[1] * lados[1]);

    printf("%s %d", "Area parcial:", area_total);
}
//@ (main)
```

Traduccion a Lisp:

```lisp
(defun main ()
(setq main_lados (make-array 3))
(setq main_area_total 0)
(setf (aref main_lados 0) 3)
(setf (aref main_lados 1) 5)
(setf (aref main_lados 2) 10)
(setf main_area_total (+ (* (aref main_lados 0) (aref main_lados 0)) (* (aref main_lados 1) (aref main_lados 1))))
(princ "Area parcial:")(princ main_area_total)
)
(main)
```

Análisis y resultados:

Al pasarlo a Lisp, se ve cómo se utiliza `make-array` para crear el tamaño del vector en memoria y luego se usan las funciones `setf` y `aref` para asignar y extraer los números de cada posición correspondiente.